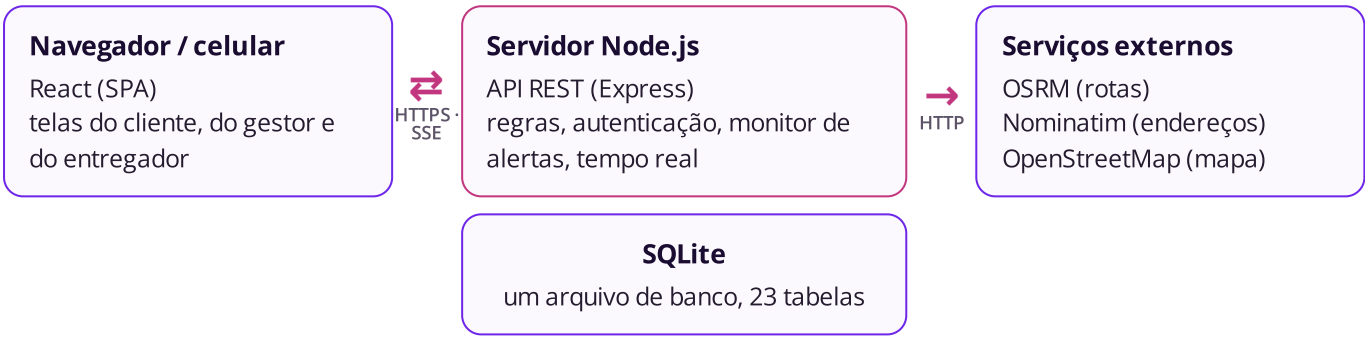
MATERIAL DE ESTUDO · APRESENTAÇÃO TÉCNICA

Rota+ — Arquitetura técnica

Linguagens, ferramentas, banco de dados, APIs, tempo real, segurança e qualidade.

1. Visão geral

O Rota+ é uma **aplicação web full stack** com arquitetura **cliente-servidor**. Em produção, um único processo Node.js entrega a API (`/api/...`) e as telas já compiladas (pasta `dist`), então o deploy é um serviço só.



O navegador conversa com o servidor por HTTPS (pedidos JSON) e recebe avisos em tempo real por SSE. O servidor guarda tudo no SQLite e consulta os serviços abertos de mapa e rota.

2. Linguagens

- **JavaScript** em todo o projeto, no front-end e no back-end: uma linguagem só para a equipe inteira.
- **SQL** para o banco de dados (`server/schema.sql`).
- **HTML e CSS** puros, sem framework de CSS; as cores ficam em variáveis em `src/styles` .

3. Front-end (roda no navegador)

Ferramenta	Para quê
React 19	Biblioteca de interface, montada com componentes reutilizáveis (botão, card, mapa…)
React Router 7	Navegação entre telas sem recarregar a página (SPA)
Vite 8	Servidor de desenvolvimento e compilação para produção
Leaflet	Mapa interativo com marcadores e trajeto
jsQR	Leitura de QR Code pela câmera, quando o navegador não tem leitor nativo
IndexedDB	Banco do próprio navegador: guarda a fila offline do entregador
Service Worker	Recebe e mostra as notificações push

Organização: `pages/` tem uma pasta por perfil (cliente, empresa, entregador, público); `components/` guarda as peças reutilizáveis; `services/api.js` concentra as chamadas ao servidor; `context/` guarda o estado global (usuário logado e dados em tempo real).

4. Back-end (o servidor)

Ferramenta	Para quê
Node.js 22+	Executa JavaScript no servidor
Express 5	Framework da API REST: rotas, middlewares e JSON
nodemailer	Envio de e-mail por SMTP
web-push	Notificações push para o celular e o navegador
qrcode	Gera o QR Code das etiquetas

Organização em camadas

- **Rotas** (`server/routes/`): os endpoints, separados por assunto: auth, entregas, ativos, alertas, localização, rotas, integração e geral.
- **Serviços** (regras de negócio): `entregasServico.js` , `alertasServico.js` , `relatorios.js` , `rota.js` .
- **Infraestrutura**: `db.js` (banco), `auth.js` (login), `tempoReal.js` (SSE), `mensageria.js` (e-mail, push, webhooks), `monitor.js` (vigia a operação a cada minuto).

5. Banco de dados

- **SQLite**, pelo módulo `node:sqlite` , que já vem embutido no Node.js: nada para instalar ou configurar.
- O banco é **um arquivo só**, com **23 tabelas**. Principais: `empresas` , `usuarios` , `sessoes` , `entregas` , `entrega_eventos` (linha do tempo), `comprovantes` , `ativos` , `ativo_movimentacoes` , `posicoes` (GPS), `alertas` , `notificacoes` , `mensagens` , `geocache` .

- **Multi-empresa:** toda tabela de dados tem `empresa_id`, e toda consulta filtra pela empresa do usuário logado (identificada pelo CNPJ).
- **Para escalar:** migrar para PostgreSQL, sem mudar as telas.

6. APIs

Externas (gratuitas e abertas)

Serviço	Para quê
OSRM	Rota real pelas ruas, melhor ordem das paradas e tempo de viagem. Sem internet, o sistema usa uma heurística própria (vizinho mais próximo + 2-opt, com distância em linha reta).
Nominatim	Transforma endereço em coordenada (geocodificação); os resultados ficam em cache na tabela `geocache`.
OpenStreetMap	As imagens do mapa.

Do próprio Rota+

- **API REST interna** (`/api/...`): usada pelas telas, autenticada por token no cabeçalho `Authorization: Bearer`.
- **API de integração** (`/api/v1/...`): outros sistemas criam e consultam entregas e ativos, autenticados por **chave de API**.
- **Webhooks**: o Rota+ avisa o sistema da empresa a cada mudança de status.
- **Importação por planilha CSV** para cadastrar entregas em lote.

7. Tempo real

Usa **Server-Sent Events (SSE)**: uma conexão aberta em que o servidor manda avisos ao navegador. O servidor só diz *o que* mudou (por exemplo, "entregas"), e a tela busca a versão nova. Se a conexão cair, a tela reconecta e ainda faz uma atualização de reserva a cada 60 s.

> **Por que SSE e não WebSocket?** A comunicação só precisa ir do servidor para o navegador. SSE é mais simples e funciona sobre HTTP comum.

8. Segurança e LGPD

- **Senhas:** hash com **scrypt** e salt aleatório; a senha nunca é guardada.
- **Sessão:** token aleatório guardado no banco, com expiração.
- **Tempo real:** ingresso de uso único, válido por 60 s, para o token de login não aparecer na URL.
- **Força bruta:** limite de tentativas por IP no login, no cadastro e no rastreio público.
- **Rastreio público:** só funciona com o código **e** o CPF/CNPJ.
- **LGPD:** GPS só no expediente da empresa, posições apagadas após 90 dias, e o cliente só vê o entregador durante a própria entrega.

9. Qualidade e ferramentas de desenvolvimento

Ferramenta	Para quê
node:test	12 testes de integração da API (`npm test`)
Playwright	9 testes de tela num navegador real, incluindo tamanho de celular (`npm run test:e2e`)
oxlint	Análise de código, hoje sem nenhum aviso
Git + GitHub	Versionamento do código
Render	Hospedagem com HTTPS e deploy automático a cada `git push`
Docker	`Dockerfile` pronto para rodar em outros provedores

10. Por que essas escolhas?

Escolha	Motivo
Só JavaScript	A mesma equipe trabalha no front e no back; o cálculo de frete é compartilhado entre os dois.
SQLite	Zero configuração para quem avalia; é SQL de verdade e migra para PostgreSQL se precisar.
OpenStreetMap e OSRM	Sem custo de licença nem chave paga, ao contrário do Google Maps.
Poucas dependências	Só 9 bibliotecas em produção: menos coisa para quebrar e manter atualizada.

Rota+ · Rastreabilidade de entregas e ativos · código em github.com/guiniwer/rota-plus · site em rota-plus-j91m.onrender.com